



# **Relacionamentos entre Classes – Associação**

**UNIVERSIDADE DO OESTE DE SANTA CATARINA - UNOESC**

**Ciência da Computação - Programação II**

Prof.: Leandro Otavio Cordova Vieira

# Objetivos da Aula

01

---

## Compreender o conceito de associação

Entender como as classes se relacionam e comunicam entre si em sistemas orientados a objetos, estabelecendo conexões que refletem o mundo real.

02

---

## Diferenciar os tipos de associação

Identificar e distinguir entre associações unidirecionais, bidirecionais e dependências, compreendendo quando aplicar cada uma.

03

---

## Praticar com exemplos concretos

Aplicar os conceitos através de exercícios práticos em PHP, desenvolvendo habilidades de implementação e modelagem de sistemas.



# Revisão Rápida da POO

## Classes

Moldes ou templates que definem a estrutura e comportamento dos objetos. Representam conceitos do mundo real através de atributos e métodos.

## Atributos

Características ou propriedades que descrevem o estado dos objetos. Armazenam dados específicos de cada instância.

## Objetos

Instâncias concretas das classes. Cada objeto possui valores específicos para os atributos definidos na classe.

## Métodos

Funções que definem o comportamento dos objetos. Permitem manipular os atributos e realizar operações específicas.

# O que é Associação?

Associação é um relacionamento estrutural fundamental entre classes que representa uma conexão lógica e duradoura entre objetos. É o mecanismo pelo qual diferentes entidades do sistema se comunicam e colaboram para realizar funcionalidades complexas, permitindo que dados e comportamentos sejam compartilhados e coordenados de maneira controlada. Este conceito é essencial para a modelagem de sistemas complexos e realistas em Programação Orientada a Objetos, onde a interdependência entre componentes é a norma.

## Definição Aprofundada

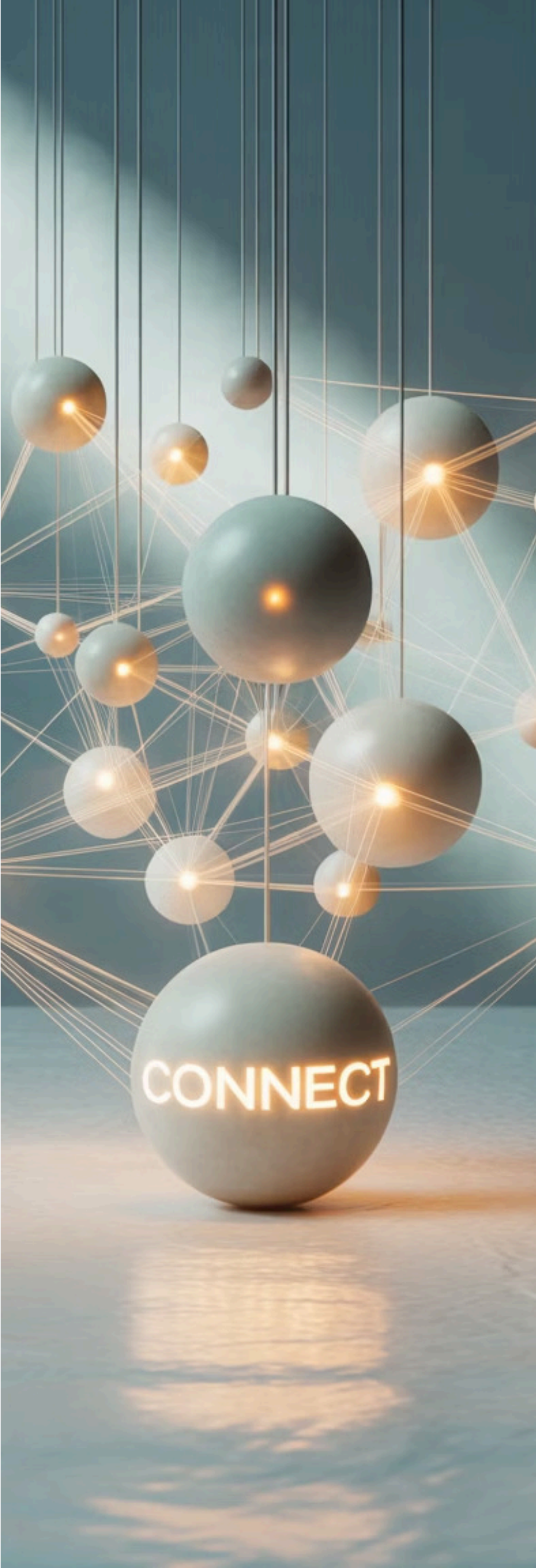
Uma associação define uma relação pela qual objetos de uma classe podem conhecer e interagir com objetos de outras classes, estabelecendo vínculos funcionais essenciais. Este vínculo pode ser unidirecional (uma classe conhece a outra, mas não vice-versa) ou bidirecional (ambas as classes se conhecem). As associações são frequentemente caracterizadas por **multiplicidade**, que especifica quantos objetos de uma classe podem se relacionar com objetos de outra (ex: um-para-um, um-para-muitos, muitos-para-muitos). Além disso, a **navegabilidade** descreve a direção na qual o relacionamento pode ser percorrido, impactando o fluxo de informação e controle no sistema.

## Importância Estratégica

As associações são cruciais para a **modularidade** do código, pois permitem que as classes sejam desenvolvidas de forma independente, mas com a capacidade de interagir de maneiras bem definidas. Elas facilitam a **manutenibilidade** do software, tornando mais fácil entender e modificar partes do sistema sem afetar drasticamente outras. Além disso, promovem a **reusabilidade**, pois classes bem definidas e seus relacionamentos podem ser empregadas em diferentes contextos ou projetos. Ao simular as interações do mundo real com fidelidade, as associações tornam o design do sistema mais intuitivo, robusto e escalável, contribuindo diretamente para a qualidade e longevidade do software.

As associações modelam uma vasta gama de relacionamentos do mundo real. Alguns exemplos incluem: um `Cliente` que faz `N` `Pedidos` (multiplicidade de 1 para N), um `Professor` que leciona `M` `Disciplinas` (N para M), ou um `Livro` que pertence a `1` `Biblioteca` (1 para 1, se não for de circulação). Outros exemplos comuns são: um `Carro` que possui `1` `Motor`, um `Departamento` que contém vários `Funcionários`, ou um `Aluno` que está matriculado em múltiplas `Turmas`. Em Programação Orientada a Objetos, estas associações são tipicamente implementadas através de referências ou ponteiros entre os objetos. Por exemplo, uma classe `Cliente` pode ter um atributo que é uma lista de objetos `Pedido`, ou um objeto `Carro` pode conter uma referência a um objeto `Motor`.

Compreender e aplicar corretamente as associações é um pilar da modelagem de dados e do design de software eficaz, garantindo que os sistemas sejam não apenas funcionais, mas também bem estruturados e fáceis de gerenciar.



# Tipos de Associação



## Associação Unidirecional

Uma classe conhece e pode acessar outra classe, mas o inverso não é verdadeiro. A navegação ocorre em apenas uma direção.



## Associação Bidirecional

Ambas as classes conhecem uma à outra, permitindo navegação em ambas as direções. Requer sincronização para manter consistência.



## Dependência (Uso)

Uma classe utiliza temporariamente outra classe, geralmente através de parâmetros de métodos. Relacionamento mais fraco e temporário.



# Representação em UML

A Linguagem de Modelagem Unificada (UML) fornece notações específicas para representar diferentes tipos de associações entre classes:

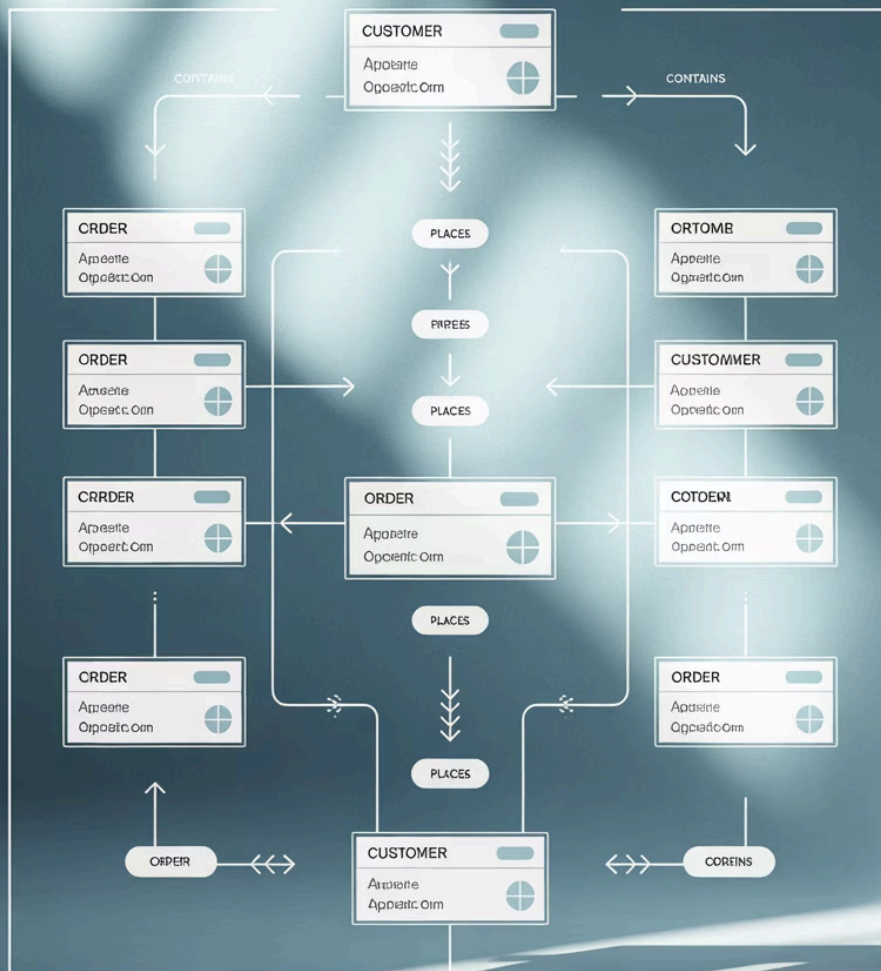
## Linha Sólida

Representa associações permanentes entre classes. Pode incluir setas para indicar direção da navegação.

As extremidades das linhas podem conter informações sobre multiplicidade (1, \*, 0..1), nomes de papéis e outras restrições que esclarecem a natureza do relacionamento.

## Linha Tracejada

Indica dependências ou relacionamentos temporários. Geralmente acompanhada da palavra "uses".



# Associação Unidirecional

Na associação unidirecional, apenas uma das classes mantém referência à outra. É o tipo mais simples de relacionamento.

## Exemplo: Aluno → Curso

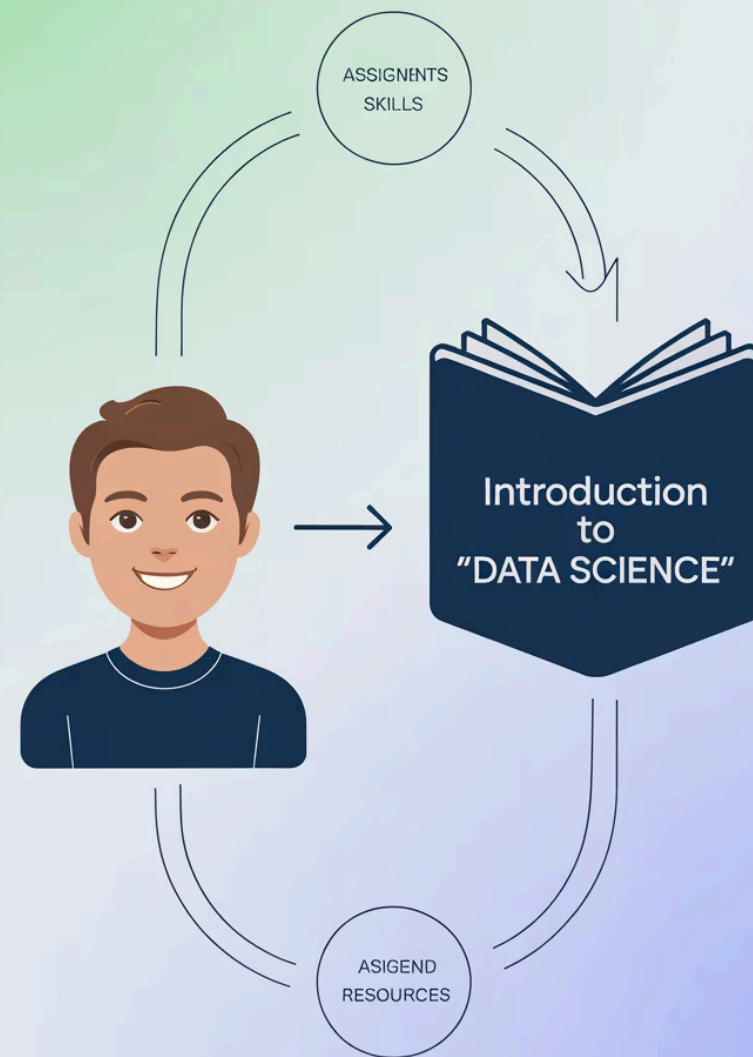
Um aluno conhece em qual curso está matriculado, mas o curso não precisa conhecer diretamente todos os seus alunos. Esta é uma relação prática onde a navegação acontece em apenas uma direção.

### Simplicidade

Fácil de implementar e manter, com responsabilidades claras entre as classes.

### Baixo Acoplamento

Reduz a dependência entre classes, facilitando mudanças e manutenção do código.



# Código em PHP – Associação Unidirecional

```
class Curso {
    private $nome;
    private $duracao;

    public function __construct($nome, $duracao) {
        $this->nome = $nome;
        $this->duracao = $duracao;
    }

    public function getNome() {
        return $this->nome;
    }
}

class Aluno {
    private $nome;
    private $curso; // Referência unidirecional

    public function __construct($nome) {
        $this->nome = $nome;
    }

    public function setCurso(Curso $curso) {
        $this->curso = $curso;
    }

    public function getCurso() {
        return $this->curso;
    }
}
```

## Análise do Código

O aluno mantém uma referência ao curso através do atributo `$curso`. O método `setCurso()` estabelece a associação, enquanto `getCurso()` permite acessar as informações do curso.

Note que a classe `Curso` não possui referência à classe `Aluno`, caracterizando a unidirecionalidade da associação.



# Associação Bidirecional

Na associação bidirecional, ambas as classes mantêm referências entre si, permitindo navegação nos dois sentidos.

## Exemplo: Pedido ↔ Cliente

Um pedido conhece qual cliente o fez, e um cliente conhece todos os seus pedidos. Esta relação mútua permite acesso completo às informações relacionadas.

### Navegação Completa

Permite acessar informações em ambas as direções, oferecendo flexibilidade na consulta de dados.

### Sincronização

Requer cuidado especial para manter a consistência dos dados em ambos os lados da associação.



# Código em PHP – Associação Bidirecional

```
class Cliente {
    private $nome;
    private $pedidos = []; // Array de pedidos

    public function __construct($nome) {
        $this->nome = $nome;
    }

    public function adicionarPedido(Pedido $pedido) {
        $this->pedidos[] = $pedido;
        $pedido->setCliente($this); // Sincronização
    }

    public function getPedidos() {
        return $this->pedidos;
    }
}

class Pedido {
    private $numero;
    private $cliente; // Referência ao cliente

    public function __construct($numero) {
        $this->numero = $numero;
    }

    public function setCliente(Cliente $cliente) {
        $this->cliente = $cliente;
    }

    public function getCliente() {
        return $this->cliente;
    }
}
```

Observe como o método `adicionarPedido()` mantém a sincronização, estabelecendo a associação em ambas as direções automaticamente.

# Dependência (Uso)

A dependência é um relacionamento mais fraco onde uma classe utiliza outra temporariamente, geralmente através de parâmetros de métodos ou variáveis locais.

## Exemplo: Professor → Turma

Um professor pode lecionar para uma turma específica durante uma aula, mas não mantém uma referência permanente a essa turma. A relação existe apenas durante a execução do método.

- **Relacionamento Temporário**

Existe apenas durante a execução de métodos específicos, não criando vínculos permanentes.

- **Baixo Acoplamento**

Minimiza as dependências entre classes, facilitando a manutenção e evolução do sistema.

- **Flexibilidade**

Permite que uma classe trabalhe com diferentes tipos sem criar vínculos permanentes.



# Código em PHP – Dependência

```
class Turma {
    private $codigo;
    private $alunos = [];

    public function __construct($codigo) {
        $this->codigo = $codigo;
    }

    public function getCodigo() {
        return $this->codigo;
    }

    public function getQuantidadeAlunos() {
        return count($this->alunos);
    }
}

class Professor {
    private $nome;

    public function __construct($nome) {
        $this->nome = $nome;
    }

    // Dependência através de parâmetro
    public function lecionar(Turma $turma) {
        echo "Professor {$this->nome} lecionando ";
        echo "para turma {$turma->getCodigo()}";
        echo " ({$turma->getQuantidadeAlunos()} alunos)";
    }
}
```

## Características

A classe Professor não mantém referência permanente à Turma. A dependência existe apenas durante a execução do método `lecionar()`.

Este padrão é ideal quando precisamos de flexibilidade, permitindo que o professor leccione para diferentes turmas sem criar vínculos permanentes.

# Cenário Real: Sistema de Vendas

Vamos analisar um sistema de vendas que demonstra diferentes tipos de associação em um contexto prático e realista.

## 1 — Pedido ↔ Produto

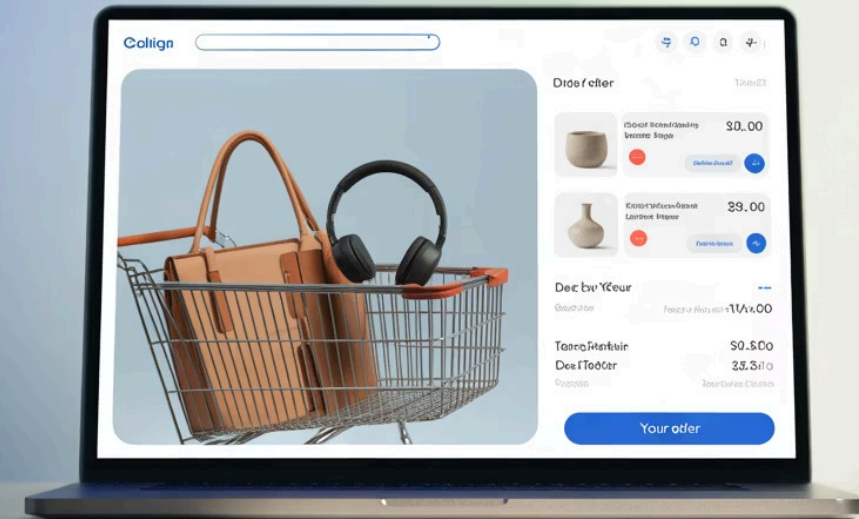
Associação bidirecional: um pedido contém vários produtos, e cada produto pode estar em múltiplos pedidos. Permite rastreamento completo de vendas e estoque.

## 2 — Pedido → Cliente

Associação unidirecional: cada pedido pertence a um cliente específico. O pedido conhece seu cliente, facilitando entregas e comunicação.

## 3 — Sistema → Relatório

Dependência: o sistema utiliza classes de relatório temporariamente para gerar análises de vendas, sem manter vínculos permanentes.





# Cenário Real: Plataforma Acadêmica

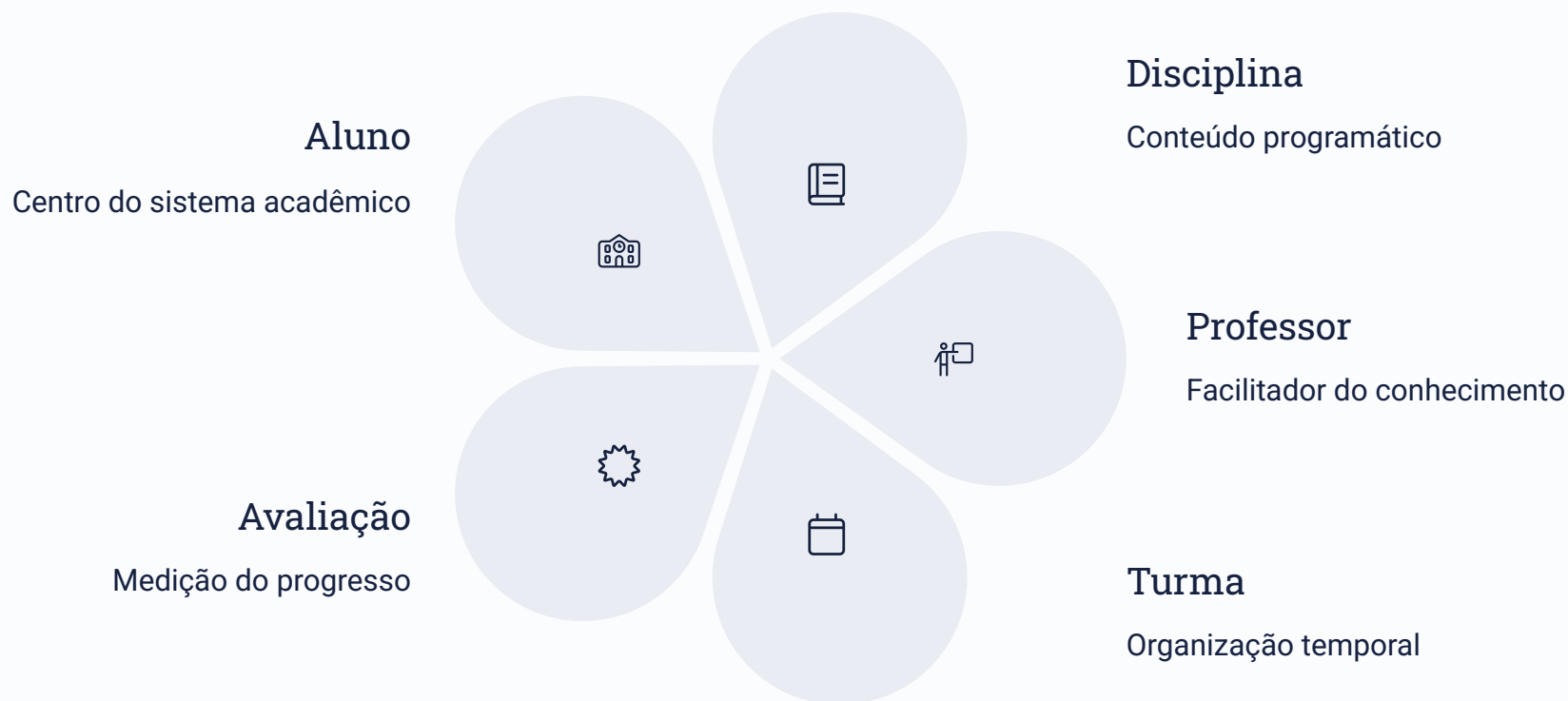
Uma plataforma acadêmica ilustra perfeitamente como diferentes tipos de associação trabalham juntos em um sistema complexo.

## Aluno ↔ Curso

Bidirecional: estudantes se matriculam em cursos, e cursos mantêm listas de alunos matriculados. Permite gestão completa de matrículas.

## Curso → Professor

Unidirecional: cada curso tem um professor responsável, mas o professor pode lecionar múltiplos cursos sem referências diretas.



# Benefícios da Associação



## Organização

Estrutura o código de forma lógica, espelhando relacionamentos do mundo real.  
Facilita a compreensão e navegação no sistema.

- Código mais intuitivo
- Estrutura espelha a realidade
- Facilita manutenção



## Clareza

Define responsabilidades claras entre classes, tornando o sistema mais compreensível para desenvolvedores e stakeholders.

- Responsabilidades bem definidas
- Interfaces claras
- Documentação natural



## Reuso

Promove a reutilização de código através de componentes bem definidos que podem ser utilizados em diferentes contextos.

- Componentes reutilizáveis
- Redução de duplicação
- Maior produtividade



# Atividade em Grupo

## Modelagem de Mini-Sistema com Associação

Agora é hora de colocar a teoria em prática! Vocês trabalharão em grupos para criar um mini-sistema que demonstre os conceitos de associação aprendidos hoje.

### Formação dos Grupos

Organizem-se em equipes de 2 a 3 alunos. Cada grupo escolherá um domínio diferente para evitar sobreposição.

### Escolha do Domínio

Selecione um sistema do mundo real: biblioteca, clínica médica, loja online, escola de idiomas, academia, etc.

### Modelagem das Classes

Identifiquem as principais entidades do sistema e criem classes com atributos e métodos relevantes.

### Implementação

Desenvolvam o código PHP com diferentes tipos de associação entre as classes criadas.





# Orientações para a Atividade

## Requisitos Obrigatórios



### Mínimo de 3 Classes

Seu sistema deve conter pelo menos três classes diferentes, cada uma representando uma entidade importante do domínio escolhido.



### Tipos de Associação

Implementem pelo menos um exemplo de cada tipo: associação unidirecional, bidirecional e dependência.



### Código Funcional

O código deve executar sem erros e demonstrar claramente os relacionamentos implementados através de métodos de teste.



### Apresentação

Preparem-se para explicar o sistema, justificando a escolha de cada tipo de associação utilizada.

**Tempo:** 60 minutos para desenvolvimento e 5 minutos por grupo para apresentação.

# Discussão e Feedback

Chegou o momento de compartilhar os resultados! Cada grupo apresentará sua solução e receberemos feedback construtivo da turma.

## CrITÉRIOS de Avaliação

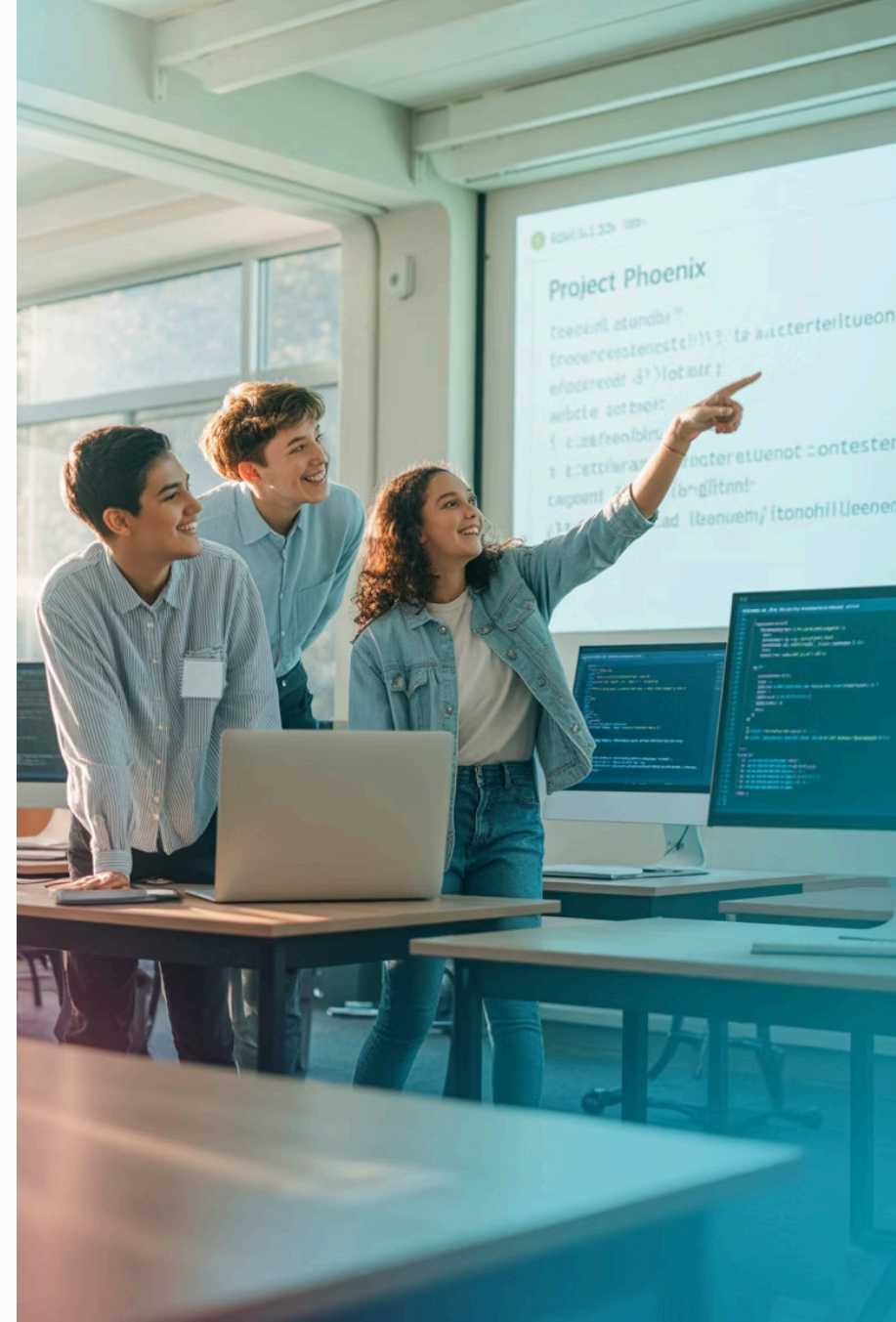
- Correção técnica da implementação
- Uso apropriado dos tipos de associação
- Clareza na explicação dos conceitos
- Criatividade na escolha do domínio
- Qualidade do código PHP

## Dinâmica da Apresentação

- 3 minutos: demonstração do sistema
- 2 minutos: explicação das associações
- 3 minutos: perguntas e feedback

Durante as apresentações, prestem atenção nas diferentes abordagens adotadas pelos colegas.

📌 Lembrem-se: o objetivo é aprender uns com os outros. Façam perguntas construtivas e compartilhem suas observações!



# Resumo da Aula

Hoje exploramos os fundamentos dos relacionamentos entre classes através da associação. Vamos recapitular os pontos principais:

## Conceito de Associação

Relacionamento estrutural entre classes que permite comunicação e colaboração entre objetos.

## Impacto no Design

Benefícios para organização, clareza e reuso de código em sistemas orientados a objetos.



## Tipos de Relacionamento

Unidirecional, bidirecional e dependência, cada um com características e aplicações específicas.

## Implementação Prática

Exemplos concretos em PHP demonstrando como implementar cada tipo de associação.

Estes conceitos são fundamentais para construir sistemas bem arquitetados e serão a base para tópicos mais avançados.



## Próxima Aula

# Agregação

Na próxima aula, avançaremos para um tipo especial de associação: a **agregação**. Exploraremos como modelar relacionamentos "parte-todo" e entenderemos a diferença entre agregação e composição.



### Relacionamento Parte-Todo

Como representar objetos compostos por outros objetos



### Notação UML

Símbolos específicos para agregação e composição



### Implementação

Código PHP demonstrando agregação na prática

**Tarefa para casa:** Revisem os conceitos de associação estudados hoje e pensem em exemplos de relacionamentos "parte-todo" que vocês conhecem no dia a dia.

